

Session 1: Grayscale Image Fundamentals

Unit Covered: Unit 1 - Display of Grayscale Images, Reading and Displaying Grayscale Images, Pixel Intensity Analysis, Basic Pixel-level Operations

Q1. Write a Python program to read and display a grayscale image. Display the following information:

- a. Image height and width
- b. Number of pixels
- c. Datatype
- d. Minimum pixel intensity
- e. Maximum pixel intensity
- f. Mean intensity
- g. Pixel value at a specified coordinate

Display the image using both OpenCV and Matplotlib.

Q2. Perform the following pixel-level operations:

- a. Increase brightness
- b. Decrease brightness
- c. Increase contrast
- d. Decrease contrast
- e. Generate negative image
- f. Apply binary thresholding

Display the original image and all the resulting images.

Challenge

Implement at least three operations using NumPy without using a ready-made OpenCV function. Explain how changing the pixel values produces the observed changes in the image.

Session 2: Image Formation, Transformation and Resampling

Units Covered: Unit 2 - Image Formation, Image Sampling, Image Quantization, Geometric Transformations, Applications of Image Zooming and Image Shrinking, Interpolation Techniques

Q1. Write a Python program to perform the following geometric transformations on an image:

- a. Translation
- b. Scaling
- c. Rotation
- d. Horizontal reflection
- e. Vertical reflection

Display the original and transformed images.

Q2. Perform image zooming and shrinking using different interpolation techniques:

- a. Nearest-neighbor interpolation
- b. Bilinear interpolation
- c. Bicubic interpolation

Perform:

- 2× zooming
- 4× zooming
- 1/2 shrinking
- 1/4 shrinking

Compare the visual quality of the resulting images.

Challenge

Implement nearest-neighbor interpolation manually using NumPy without using a ready-made image resizing function. Explain why different interpolation methods produce different image quality.

Session 3: Mathematical Foundations of Image Processing

Units Covered: Unit 3 - Pixel Relationships, Connectivity, Adjacency, Distance Measures, 2-D Discrete Fourier Transform (DFT), 2-D Discrete Cosine Transform (DCT)

Q1. Write a Python program to investigate the spatial relationship between pixels in a binary image.

Perform:

- a. 4-neighbour connectivity
- b. 8-neighbour connectivity
- c. m-connectivity
- d. Euclidean distance
- e. City-block distance
- f. Chessboard distance

Demonstrate the results using a suitable binary image.

Q2. Perform the following frequency-transform operations on a grayscale image:

- a. Calculate the 2-D Discrete Fourier Transform (DFT).
- b. Shift the zero-frequency component to the center.
- c. Calculate and display the magnitude spectrum.
- d. Apply logarithmic scaling to the spectrum.
- e. Calculate the 2-D Discrete Cosine Transform (DCT).
- f. Display the DCT coefficient matrix.

Challenge

Retain only a selected percentage of the low-frequency DCT coefficients and reconstruct the image using inverse DCT. Compare the reconstructed image with the original and explain the observed loss of detail.

Session 4: Histogram Processing and Contrast Enhancement

Units Covered: Unit 4 - Histogram Processing, Image Histograms, Cumulative Distribution Function (CDF), Contrast Enhancement, Linear Contrast Stretching, Non-linear Contrast Stretching, Histogram Equalization

Q1. Write a Python program to calculate and display the histogram and cumulative distribution function (CDF) of a grayscale image.

Perform the analysis for:

- a. A dark image
- b. A bright image
- c. A low-contrast image
- d. A high-contrast image

Explain what the histogram reveals about each image.

Q2. Perform the following contrast-enhancement operations:

- a. Linear contrast stretching
- b. Non-linear contrast stretching
- c. Histogram equalization

For each technique, display:

- Original image
- Enhanced image
- Original histogram
- Enhanced histogram

Challenge

Implement histogram equalization manually using the CDF and compare your result with OpenCV's `equalizeHist()` function. Determine whether histogram equalization always produces a better image.

Session 5: Spatial-Domain Filtering and Non-linear Filtering

Units Covered: Unit 4 - Spatial Domain Method, Spatial Filtering, Smoothing, Image Averaging, Mean Filter, Gaussian Filtering, Low-pass Filtering, Non-linear Filtering, Median Filtering, Minimum Filtering, Maximum Filtering

Q1. Write a Python program to add noise to a grayscale image and perform spatial-domain smoothing using:

- a. Image averaging
- b. Mean filter
- c. Gaussian filter
- d. Low-pass filter

Experiment with different kernel sizes and compare the results.

Q2. Apply the following non-linear filters to a noisy image:

- a. Median filter
- b. Minimum filter
- c. Maximum filter

Test the filters using different kernel sizes and determine which filter is most suitable for removing salt-and-pepper noise.

Challenge

Implement a 3×3 median filter manually using NumPy without using a ready-made filtering function. Compare the manually generated result with the OpenCV result and explain the difference, if any.

Session 6: Image Sharpening and Edge Detection

Units Covered: Unit 4 - Image Sharpening, High-pass Filtering, High-boost Filtering, Derivative Filtering, First-Order Derivatives, Second-Order Derivatives, Edge Detection Using Roberts, Prewitt, Sobel and Laplacian Operators

Q1. Write a Python program to enhance image details using spatial-domain sharpening techniques:

- a. High-pass filtering
- b. Laplacian filtering
- c. Unsharp masking
- d. High-boost filtering

Display and compare the resulting images.

Q2. Detect edges in a grayscale image using:

- a. Roberts operator
- b. Prewitt operator
- c. Sobel operator
- d. Laplacian operator
- e. Canny edge detector

Compare the edge maps produced by the different operators.

Challenge

Implement either the Prewitt or Sobel operator manually using convolution. Apply it to both a clean image and a noisy image and explain why noise affects edge detection.

Session 7: Frequency-Domain Image Enhancement

Units Covered: Unit 4 - Frequency Domain Method, Image Enhancement in the Frequency Domain, 2-D DFT, Frequency-Domain Low-pass Filtering, Frequency-Domain High-pass Filtering, Ideal Filters, Gaussian Filters, Homomorphic Filtering

Q1. Write a Python program to perform frequency-domain filtering using:

- a. Ideal Low-pass Filter
- b. Gaussian Low-pass Filter
- c. Ideal High-pass Filter
- d. Gaussian High-pass Filter

For each filter:

- Calculate the DFT.
- Shift the frequency spectrum.
- Generate the filter mask.
- Apply the mask.
- Perform inverse DFT.
- Display the resulting image.

Q2. Apply homomorphic filtering to an image affected by uneven illumination.

Perform:

- a. Log transformation
- b. Fourier transformation
- c. Construction of a homomorphic filter
- d. Frequency-domain filtering
- e. Inverse Fourier transformation
- f. Exponential transformation

Compare the original and enhanced images.

Challenge

Create a program that allows the user to change the cut-off frequency of a frequency-domain filter. Investigate how different cut-off frequencies affect image smoothness, sharpness and detail.

Session 8: Image Restoration and Color Image Processing

Units Covered: Unit 5 - Image Restoration, Image Degradation Models, Noise and Blur Models, Inverse Filtering, Wiener Filtering, Least-Square Restoration, Display of Color Images, RGB Color Model, HSV Color Model, Color Channel Processing

Q1. Create a degraded image by introducing suitable noise and/or blur.

Perform image restoration using appropriate techniques such as:

- a. Inverse filtering
- b. Wiener filtering
- c. Least-square restoration

Compare:

- Original image
- Degraded image
- Restored image

Calculate MSE and PSNR to evaluate the restoration quality.

Q2. Read and display a color image and perform the following operations:

- a. Separate the Red, Green and Blue channels.
- b. Display each channel separately.
- c. Modify individual color channels.
- d. Reconstruct the color image.
- e. Convert the image into grayscale.
- f. Convert the image into HSV color space.

Challenge

Use the HSV representation to detect and extract a selected color from an image. Explain why HSV can be more useful than RGB for color-based processing.

Session 9: Discrete Wavelet Transform of Images

Units Covered: Unit 5 - Discrete Wavelet Transform (DWT) of Images, One-Level 2-D DWT, Multi-Level DWT Decomposition, LL, LH, HL and HH Sub-bands, Inverse DWT Reconstruction, Wavelet-Based Image Analysis

Q1. Write a Python program to perform a one-level 2-D Discrete Wavelet Transform on a grayscale image.

Generate and display:

- a. LL component
- b. LH component
- c. HL component
- d. HH component

Explain what type of image information is represented by each component.

Q2. Perform multi-level DWT decomposition and image reconstruction.

Perform:

- a. One-level decomposition
- b. Two-level decomposition
- c. Three-level decomposition
- d. Inverse DWT reconstruction

Compare the reconstructed image with the original image.

Challenge

Remove selected high-frequency wavelet coefficients and reconstruct the image. Observe and explain the effect on image details, edges and overall image quality.

Session 10: Point, Line and Advanced Edge Detection

Units Covered: Unit 6 - Point Detection, Line Detection, Edge Detection, Combined Point-Line-Edge Detection, Edge Linking, Boundary Detection, Hough Transform, Hough Line Transform, Hough Circle Transform

Q1. Write a Python program to detect significant points and lines in an image.

Perform:

- a. Point detection
- b. Horizontal line detection
- c. Vertical line detection
- d. Diagonal line detection

Display the detected features over the original image.

Q2. Perform advanced edge and boundary detection using:

- a. Canny edge detection
- b. Edge linking
- c. Hough Line Transform
- d. Hough Circle Transform

Display the detected lines/circles over the original image.

Challenge

Develop a program that detects and counts a particular geometric object, such as lines or circles, in an unknown image using Hough Transform. Explain how the threshold parameters affect detection.

Session 11: Image Segmentation and Watershed Transform

Units Covered: Unit 6 - Image Segmentation, Thresholding, Global Processing, Optimal Thresholding, Adaptive Thresholding, Region-Oriented Segmentation, Region Growing, Region Splitting and Merging, Watershed Segmentation, Marker-Based Segmentation, Boundary Detection

Q1. Segment a grayscale image using different thresholding and region-based techniques:

- a. Simple global thresholding
- b. Optimal thresholding
- c. Adaptive thresholding
- d. Region growing

Compare the segmentation results.

Q2. Perform watershed-based segmentation on an image containing multiple touching objects.

Perform:

- a. Image preprocessing
- b. Thresholding
- c. Morphological noise removal
- d. Distance transformation
- e. Foreground marker generation
- f. Background marker generation
- g. Watershed transformation
- h. Boundary detection

Challenge

Use watershed segmentation to separate and count touching objects such as coins, cells or particles. Compare the result with simple thresholding and explain why watershed provides better separation.

Session 12: Image Compression and Integrated Image Processing

Units Covered: Unit 6 - Image Compression, Lossless Compression, Lossy Compression, JPEG Compression, DCT-Based Image Compression, DWT-Based Image Processing, Compression Ratio, MSE, PSNR, Applications of Image Zooming and Image Shrinking, Integrated Image-Processing Pipeline

Q1. Investigate lossy and lossless image compression.

For a given image:

- a. Save the image in PNG format.
- b. Save the image in JPEG format with quality 90.
- c. Save the image in JPEG format with quality 70.
- d. Save the image in JPEG format with quality 50.
- e. Save the image in JPEG format with quality 20.

For each version, calculate:

- File size
- Compression ratio
- MSE
- PSNR

Compare the visual quality of the compressed images.

Q2. Implement a simplified DCT-based image compression system.

Perform:

- a. Divide the image into 8×8 blocks.
- b. Apply DCT to each block.
- c. Reduce/quantize selected coefficients.
- d. Apply inverse DCT.
- e. Reconstruct the image.
- f. Compare the reconstructed image with the original.
- g. Calculate MSE, PSNR and compression ratio.

Challenge

Given an unfamiliar image, design a complete image-processing pipeline to solve a practical problem. Select appropriate techniques from the experiments learned throughout the course, such as:

Image acquisition → Grayscale/Color processing → Enhancement → Noise removal → Edge detection → Segmentation → Compression/Resizing